Embedded Computer Vision Compression Techniques for Soccer Analysis

Jack Coleman

The thesis is submitted to University College Dublin in part fulfilment of the requirements for the degree of Master of Electronic and Computer Engineering.

School of Electrical and Electronic Engineering

Supervisor: Deepu John

April 2026

Contents

1 Abstract **1**

2 Lay Abstract **2**

3 Introduction **2**

4 Problem Aims **3**
4.1 Model Training and System Creation . . . 3
4.2 Optimization . . . 4
4.3 Compression . . . 4

5 Background Literature Review **4**
5.1 Literature Review: Overview . . . 5
5.2 Literature Review: Common Approaches . . . 5
5.2.1 Common Approaches: Multiple Object Tracking . . . 6
5.2.2 Common Approaches: Key-point Detection . . . 7
5.3 Literature Review: Current Limitations . . . 8
5.4 Literature Review: Compression Techniques . . . 9
5.4.1 Compression Techniques: Quantization . . . 9
5.4.2 Compression Techniques: Pruning . . . 10
5.4.3 Efficiency Techniques: Hardware Acceleration . . . 11
5.4.4 Compression Techniques: Knowledge Distillation . . . 12

6 Methodology **12**
6.1 Model Training Procedure . . . 12
6.2 Compression . . . 13
6.3 Hardware Acceleration . . . 14
6.3.1 Quantisation . . . 14
6.3.2 Pruning . . . 15
6.3.3 Knowledge Distillation . . . 15
6.4 Evaluation Metrics and Benchmarking Protocol . . . 16
6.5 Post-Processing and Higher-level Statistic Generation . . . 17
6.6 Full-Stack System Overview . . . 18

7 Results **19**
7.1 Selecting Model Architecture and Dataset . . . 19
7.2 Summary of Model Training . . . 20
7.3 Compression Analysis . . . 20
7.3.1 Hardware Acceleration . . . 23
7.3.2 Quantisation . . . 24
7.3.3 Pruning . . . 25
7.3.4 Knowledge Distillation . . . 26
7.3.5 Combinations of Different Techniques . . . 27
7.4 Hardware Analysis . . . 28
7.4.1 Latency . . . 28
7.4.2 GPU Usage . . . 28
7.4.3 CPU Usage . . . 29
7.4.4 Power Draw . . . 30

7.4.5 Temperature 31
7.5 Full-Stack System 31
7.5.1 Full-Stack System Performance 31

8 Impact of Research **32**
8.1 Ethical Considerations 32
8.2 Sustainability Considerations 33

9 Proposed Future Work **33**
9.1 System Functionality Improvements 34
9.2 System Optimization 34
9.3 Model Compression 34

10 appendices **42**

1 Abstract

This thesis investigates how embedded computer vision models for soccer analysis can be trained, compressed, and deployed efficiently on edge hardware. The work focuses on object detection and pitch keypoint localisation using YOLO11 and YOLO26 model families, with benchmarking carried out on an NVIDIA Jetson Orin Nano Dev Kit. After an initial architecture and dataset selection study, a full experimental pipeline was developed covering baseline training, hardware acceleration, quantisation, pruning, knowledge distillation, and combinations of these techniques. In total, 488 model artefacts were generated and evaluated across the object detection and pose estimation tasks.

The results show that hardware acceleration preserves analytical performance almost exactly while delivering substantial latency reductions, with average improvements of 31.3% for object detection and 29.9% for pose estimation. Quantisation produced the largest deployment gains, reducing latency by an average of 56.6% for object models and 59.6% for pose models, while also giving the strongest reduction in power usage. However, these benefits came with a larger accuracy cost, particularly for pose estimation. Pruning provided moderate latency improvements for object detection but caused much larger accuracy degradation in the pose task, indicating that the keypoint models were less robust to parameter removal. Knowledge distillation on its own had only a minor effect on latency, but remained useful as a supporting technique for preserving accuracy in smaller deployable models.

These compressed models were then integrated into a full-stack soccer analysis system for player and ball detection, pitch localisation, team assignment, possession estimation, and pass detection. Under the strongest replay setting, the best full-stack configuration achieved a Pass $mAP_{1\text{–}5}$ of 0.697 and 39.61% GPU utilisation, while reaching an effective throughput of 13.02 FPS on the target device. Overall, the thesis shows that deployment-focused compression can substantially improve the practicality of embedded soccer analytics while retaining useful analytical performance, and that a full computer vision pipeline for match statistic generation can be run effectively on constrained edge hardware.

2 Lay Abstract

This thesis looks at how soccer video analysis can be run on a small, low-power computer instead of relying on expensive cloud or server hardware. The work trained many versions of software that can find players, the ball, and the pitch in match footage, and then tested different ways of making that software smaller, faster, and cheaper to run. These included speeding up the software for the target device, reducing numerical precision, removing less useful parts of the models, and using larger models to help train smaller ones.

The study found that some of these methods gave large speed and power improvements while still keeping the analysis accurate enough to remain useful. The most effective overall approach was to combine efficient deployment methods with carefully chosen compressed models, rather than applying the most aggressive simplifications everywhere.

The final system was able to combine these models into a complete match-analysis pipeline that estimates team possession and detects passes from broadcast video. The best version of the full system reached an effective speed of about 13 frames per second on the target edge device, while keeping GPU usage below 40%. This shows that practical, on-device soccer analytics is possible on compact hardware, which could help smaller teams, clubs, and broadcasters gain access to lower-cost performance analysis tools.

3 Introduction

The use of advanced analytical methods in sport gained popularity in the mid-1990s through sabermetric principles in professional baseball [12]. Since then, data-driven analysis has become central to performance evaluation, tactical decision-making, and modern sports broadcasting. During live coverage, high-level statistics such as possession, shot counts, and pass completion rates are commonly shown alongside the scoreline and elapsed time. Historically, however, the collection of this data has typically been outsourced to specialised and expensive third-party analytics providers. For example, major English and Scottish football leagues have long-term data rights agreements with Opta, granting exclusive responsibility for the collection and distribution of official match statistics across

broadcast, media, and analytical platforms [35].

More recently, the growth of online streaming platforms has increased the broadcast coverage of lower-budget and regional sporting events [26]. At the same time, amateur and semi-professional teams are increasingly adopting data-driven approaches to support training and performance evaluation [11]. Advances in computer vision now make it possible to analyse sports video streams in real time, enabling low-cost, on-site generation of match statistics without reliance on cloud infrastructure. This could reduce costs for both sports broadcasting and sports science applications, with particular benefits for lower-budget organisations.

These trends motivate the aim of this work: **the optimisation and compression analysis of embedded computer vision models for real-time soccer analysis on an edge device.**

4 Problem Aims

4.1 Model Training and System Creation

The first and most basic objective of this project is to train computer-vision models that can automatically extract low-level information from video of soccer games. These models should be trained with the intention of being run on an edge device, but should have an mAP_{50} within 10% to that of other modern, non real-time approaches [3]. Mean Average Precision (mAP) is a common metric for object detection models that measures the accuracy of predicted bounding boxes by combining precision and recall across different confidence thresholds. mAP_{50} specifically refers to the mAP computed using an Intersection over Union (IoU) threshold of 0.50, meaning a predicted bounding box is only counted as a correct detection if it overlaps the ground-truth box by at least 50%.

For this project, an NVIDIA Jetson Orin Nano Dev Kit will be used as the target edge device [23]. Alongside these models, an end-to-end system should be developed, to convert the video and model outputs to desired high-level game statistics. This will allow the

functionality of the system to be explored, as well as assess hardware and throughput performance of the edge device.

4.2 Optimization

The second objective is to optimise the end-to-end system with respect to computational efficiency, aiming to reduce CPU, GPU, and memory utilisation, device temperature and power draw without significantly affecting analytical performance. Specifically, the system should perform to within 15% mAP_{50} of modern object detectors [3] used for soccer analysis, while utilizing no more than 80% of the device's total CPU, GPU, and RAM availability. Similarly, the power draw and temperature should not exceed 80% of the device's recommended limits, and maintain roughly 20 FPS to allow for real-time analysis. NVIDIA recommends a maximum CPU and GPU temperature of 85° and a max power draw of 25W, and so these will used as the initial recommended limits [23].

4.3 Compression

The third objective is to apply model-level compression techniques directly to the trained models. As with before, compressed models should maintain an analytical performance within 15% mAP_{50} of the baseline uncompressed models, while targetting a FLOP (Floating Point Operations Per Second) reduction of 5x and a parameter size reduction of 5x. This will also indirectly lower GPU/CPU and power requirements, supporting the 80% total CPU, GPU, and RAM availability aims discussed earlier.

5 Background Literature Review

To achieve these aims, a background literature review will be performed to ensure that the proposed methodology described later utilises best modern practices and research, and targets a clear gap in research. Focus will be placed on an overview of the problem, common approaches and prevalent models, and common model compression techniques.

5.1 Literature Review: Overview

As discussed in the introduction, modern sporting preparation and broadcast have put a larger emphasis on quantitative data than opinion-based analysis, especially with the rise of next-generation tracking and sensor technologies [29]. Some of the most useful of these implementations explore tracking key statistics such as team pass completions percentages, possession, and team formation analysis through the use of object detection/tracking and pitch recognition [44, 20]. However, emphasis in this niche has been largely placed on accuracy over computational efficiency or real-time analysis, with some designed software systems requiring over 515B Floating Point Operations Per Second (FLOPS) [31], which edge devices cannot maintain.

5.2 Literature Review: Common Approaches

Modern computer vision-based approaches to estimating possession and formation often utilise object detection models coupled with object tracking algorithms to produce Multiple Object Tracking (MOT) outputs [3]. This allows identities (in the form of bounding boxes) for on-screen outfield players, goalkeepers, referees, and the ball to be used for statistic generation. From here, clustering techniques such as K-means and mean shift methods are often applied, separating detecting players into two teams, as well as determining the goal-keepers [45]. This team assignment allows detailed estimates of possession and pass completion to be determined.

Alongside object detection for players, a variety of different key-point detection strategies are often used to detect the location on screen of relevant pitch markings, such as the corners, penalty box, and midfield [8]. With this and the player bounding boxes from earlier, an optical perspective transformation known as homography estimation can be applied to calculate the co-ordinate location on the pitch of each object of interest [5]. This allows player speed, distance, and position statistics to be generated.

The reported features and statistics of some implemented software solutions are displayed below in Table 1. Note that reported FPS values are taken from original implementa-

tions, and are not directly comparable across hardware. Also note that model accuracy and size values (mAP_{50}, FLOPS, Parameters) cannot be explored here, due to significant differences in software structures (both in amount of models, model output information, and implementation output). Instead, the accuracy and size of the models used in these will be explored explicitly in subsections **Multiple Object Tracking** and **Key-point Detection** later.

Implementation	Primary Models Used	FPS	Features	Real-time?
RoboFlow [31]	YOLOv10m-pose (Pitch) YOLOv8x (Player) YOLOv8x (Ball)	< 1	Position Map	No
CodeJiffy [21]	YOLOv11n-pose (Pitch) YOLOv11s (Players, Ball)	~ 7	Position Map, Possession Speed Estimation	No
SoccerMaster [40]	QWEN2.5 (Jerseys) YOLOv8 (Players)	< 1	Event Detection, Position Map Jersey Number Detection	No
HMBZO [10]	YOLOv8m (Pitch) YOLOv8l (Players, Ball)	~ 9	Position Map	Yes

Table 1: Previously Implemented Football Analysis Software Solutions

Note: Accuracy and size comparisons (mAP_{50}, FLOPS, Parameters) are omitted due to architectural differences.

5.2.1 Common Approaches: Multiple Object Tracking

The object detection portion of the above solutions are commonly addressed using YOLO (You Only Look Once) models [27]. YOLO models utilise a single pass for both localisation and detection, resulting in a low-latency detection architecture ideal for real-time analysis. This has been previously explored in soccer analysis, with models producing mAP_{50} results of up to 87%, and mAP_{50-95} of up to 64.9% using labelled broadcast soccer videos [18]. mAP_{50-95} represents the model's mAP for all IoU thresholds between 50%-95%. Another potential real-time detection model is the Faster R-CNN model. Faster R-CNN utilises a 2-stage pipeline, which first separates frames into region proposal networks, before performing classification on a second pass [30]. While used in post-match processing, Faster R-CNN is likely not applicable to real-time scenarios, due to this heavy 2-stage inference process. Table 2 below summarizes some commonly used models for real-time object detection. Note

that latency cannot be compared directly here, as models have not been benchmarked on equal GPUs.

Model	**AP$_{50-95}$** (%)	**Parms** (M)	**FLOPS** (B)
YOLO [27, 37]			
v11-n	39.5	2.6	6.5
v11-s	47.0	9.4	21.5
v11-m	51.5	20.1	68.0
v8-n	37.3	3.2	8.7
v8-s	44.9	11.2	28.6
v10-s	46.8	N/A	21.6
RT-DETR-R18 [43]	46.5	19.2	60
R50-FPN [30, 15]	41.0	60	160

Table 2: Commonly Used Object Detection Models

A number of commonly used and publicly available datasets exist for the training of these models. The most popular and utilised datasets are displayed below.

SoccerNet Dataset [3]

Class	**Bounding Boxes**
Player	2,992,173
Goalkeeper	130,109
Referee	301,025
Ball	215,156
Total	**3,645,661**

Roboflow Dataset [32]

Class	**Bounding Boxes**
Player	13,241
Goalkeeper	473
Referee	1,519
Ball	565
Total	**15,798**

Table 3: Commonly Used Datasets in Literature

The tracking of these bounding boxes is then implemented (MOT); modern approaches often use the SORT algorithm, in which object identities are maintained using Kalman filters for motion prediction and the Hungarian algorithm for object association [1].

5.2.2 Common Approaches: Key-point Detection

As discussed earlier, on-screen locations of key pitch marking are desired to allow homography estimation to be applied. Computer vision models are commonly used for this as well. Offline systems often use encoder–decoder architectures, which estimate pitch key locations by compressing image features into a latent representation and then decoding this

representation to produce precise key-point predictions [2]. More recently, YOLO-based pose detection models have been applied [17], utilising their single pass architecture to enable low-latency key-point estimation. Figure 4 displays some models that will be initially explored. As with before, latency comparisons are not feasible due to benchmarking differences between literature.

Model	**mAP**$_{50-95}$	**Params (M)**	**FLOPS (B)**
YOLO-Pose [27, 37]			
v11n-pose	50.0	2.9	7.4
v11s-pose	58.9	9.9	23.1
v11m-pose	64.9	20.9	71.4
v8n-pose	48.7	3.3	8.9
v8s-pose	56.8	11.6	30.2
HRNet-W32 [39]	66.4	28.5	95.0
OpenPose-Light [25]	54.0	25.9	160.0

Table 4: Keypoint Detection Models

Some classical, non-neural approaches also exist, which align on-screen edges with known pitch dimensions to project player positions into real-world coordinates without learned representations [16]. While this enables significantly faster throughput and avoids the need for training data, performance is often sensitive to occlusion, inaccurate line detection, and changes in broadcast viewpoint or visual clutter, limiting robustness in real match conditions.

5.3 Literature Review: Current Limitations

Despite the rapid progress in computer vision for sports, several significant technical and operational hurdles remain unexplored for real-time edge deployment. As Table 1 displayed earlier, common models used today [10, 40, 21, 31] are computationally intensive, demanding high-end GPU resources (e.g, NVIDIA A100 TensorRT) that are typically unavailable on constrained edge devices like the Jetson Nano. Most systems are designed for batch frame processing, and require 200+ of GFLOPS to produce < 1 FPS in real-time [40, 31]. Other systems, while feasible for real time, assume much more powerful infrastructure and still produce only 5-10 FPS on consumer hardware[10, 21]. This leads to

systems and software unsuitable for edge deployment without first applying optimisation or compression.

5.4 Literature Review: Compression Techniques

A range of model compression techniques designed to reduce the computational and memory requirements of deep learning models (while preserving acceptable analytical performance) have been explored in literature. Although these techniques have not yet been widely applied to end-to-end football statistic generation systems, they will form a strong foundation for deploying real-time soccer analysis pipelines on edge devices.

5.4.1 Compression Techniques: Quantization

Quantization is one of the most widely adopted compression techniques, and involves reducing the numerical precision of model parameters and activations [7]. While standard, uncompressed neural networks typically utilise 32-bit floating-point (FP32) representations, quantization maps these values to lower-precision formats such as 8-bit (INT8) or 4-bit (INT4) integers. This process substantially reduces memory bandwidth usage and arithmetic complexity, with INT8 addition being up to 30× more energy-efficient and 116× more area-efficient than its FP32 counterpart on 45nm hardware [7]. Figure 1 below displays the effect of different quantization levels on device power consumption.

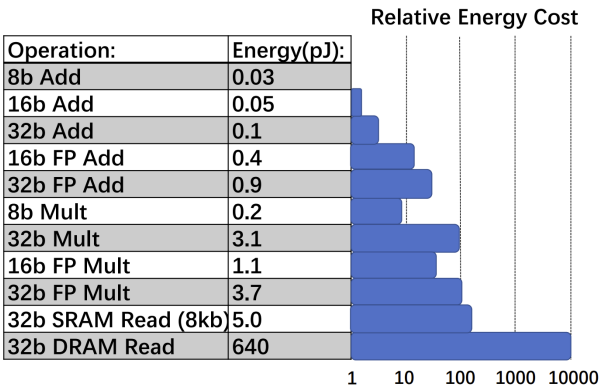


Figure 1: Effect of Quantization of Model Energy Consumption [7]

As can be seen, identical operations utilise significantly less energy (in Joules) when using less precise values. For example, an 8-bit multiplication requires 0.2 pJ, while a 32b

multiplication requires 3.1 pJ. This demonstrates the clear optimization benefit that quantization can supply.

5.4.2 Compression Techniques: Pruning

The technique of pruning aims to reduce model complexity by removing parameters that contribute minimally to the network's output [7, 41]. This technique follows the assumption that the smallest system capable of fitting the training data typically yields the best generalization on unseen inputs [28]. It is often performed after training, where a loss function is used to estimate the importance of weights, before identifying and eliminating redundant connections.

Pruning strategies are fundamentally divided into unstructured and structured approaches. Unstructured pruning removes individual weights with low importance regardless of their location [7], which achieves high compression ratios at the cost of efficient memory manipulation. Structured pruning removes whole structures such as entire filters, channels, or layers, which maintains efficient memory at the cost of less compression. [41]. Figure 2 below displays model accuracy (TOP-1) and FLOPS, when various pruning strategies are applied.

	Method	Top-1	FLOPs	Top-1 ↓	FLOPs ↓ (%)
Swin-T	Baseline	81.2	4.5	0.0	0.0
	STEP	77.2	3.5	4.0	22.2
	ViT-Slim	80.7	3.4	0.5	24.4
	X-Pruner (Ours)	**80.7**	**3.2**	**0.5**	**28.9**
Swin-S	Baseline	83.2	8.7	0.0	0.0
	STEP	79.6	6.3	3.6	27.6
	WDPruning	81.8	6.3	1.4	27.6
	X-Pruner (Ours)	**82.0**	**6.0**	**1.2**	**31.0**

Figure 2: Effect of Pruning on Accuracy and FLOPS [41]

As can be seen, applying the X-Pruner algorithm reduces the required FLOPS by 28%, with only a .5% reduction in accuracy (Top-1). For this reason, pruning will be explored as a method of compressing training soccer computer vision models.

5.4.3 Efficiency Techniques: Hardware Acceleration

Hardware acceleration improves inference efficiency by executing neural networks through runtimes and software stacks that are specifically optimized for the target hardware platform [46]. Rather than relying on a default framework execution path, accelerated backends can apply graph-level optimizations, fuse operations, select more efficient kernels, and better exploit GPU parallelism. This is particularly important in real-time computer vision, where inference latency directly affects whether a system can operate at interactive or live processing speeds.

A common approach to hardware acceleration in embedded and GPU-based deep learning systems is the use of optimized inference runtimes such as TensorRT. These frameworks convert trained models into deployment-ready representations that are tailored to the hardware on which inference will be performed. By moving much of the optimization work offline, such approaches reduce runtime overhead and improve throughput, making them especially valuable in edge settings where compute and power budgets are constrained.

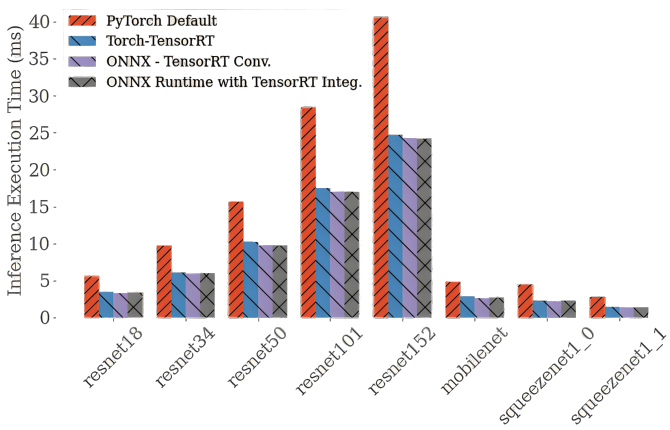


Figure 3: Effect of `tensorrt` application on inference time.

Figure 3 illustrates the typical effect of hardware acceleration on inference execution time [46]. As can be seen, optimized TensorRT-based pipelines consistently reduce latency relative to the default PyTorch implementation, with similar trends also observed for ONNX-based accelerated deployment. This demonstrates that hardware acceleration can provide substantial efficiency improvements without altering the underlying model task, making it

an important optimization strategy in the deployment of real-time computer vision systems.

5.4.4 Compression Techniques: Knowledge Distillation

Knowledge distillation is another model compression technique, in which a compact "student" model is trained to replicate the performance of a high-capacity "teacher" model. Unlike standard training, the student is optimised using softened output distributions (logits) or intermediate feature representations produced by the teacher, which contain richer information than hard ground-truth labels [9, 7]. Knowledge transfer can be categorised into three types: response-based, which focuses on the final output probabilities; feature-based, which aims to mimic the teacher's internal feature maps to capture implicit patterns; and relation-based, which teaches the student the relationships between different data samples or layers [9].

6 Methodology

The methodology will be focussed into three parts; the training of initial models, the compression of these models, and the creation and optimisation of a full-stack system to utilise these models to apply high-level analysis of broadcast soccer film.

6.1 Model Training Procedure

To support fair comparison, all uncompressed object detection and keypoint detection models were trained under a standard procedure. Training was initialised from COCO-pretrained weights [14], and then fine-tuned on a football-targeted dataset. All models were trained in PyTorch and saved as `.pt` checkpoints prior to compression. Training used an input resolution of 1280×1280, a batch size of 32, and was conducted on the UCD high-performance cluster *UCDSonic*. Optimisation used stochastic gradient descent with an initial learning rate of 0.01, momentum 0.937, and weight decay 5×10^{-4} [38]. Training ran for up to 500 epochs, with a 3-epoch warm-up schedule and early stopping patience of 60 epochs.

An initial benchmarking study was performed to identify the most suitable model family for the remainder of the work, based on analytical performance and hardware usage. A separate preliminary study was then conducted to determine the most suitable dataset for the full object detection study, in which 10 model variants were trained under identical conditions and evaluated across three labelled football broadcast datasets: SoccerNet [3], Roboflow [31], and Football Analytics [33]. For pitch keypoint detection, the Roboflow football field dataset [34] was used throughout.

6.2 Compression

As discussed in the Literature Review, three chief compression techniques have been explored: quantisation, knowledge distillation, and pruning. Figure 4 below displays the various combinations of compression techniques applied.

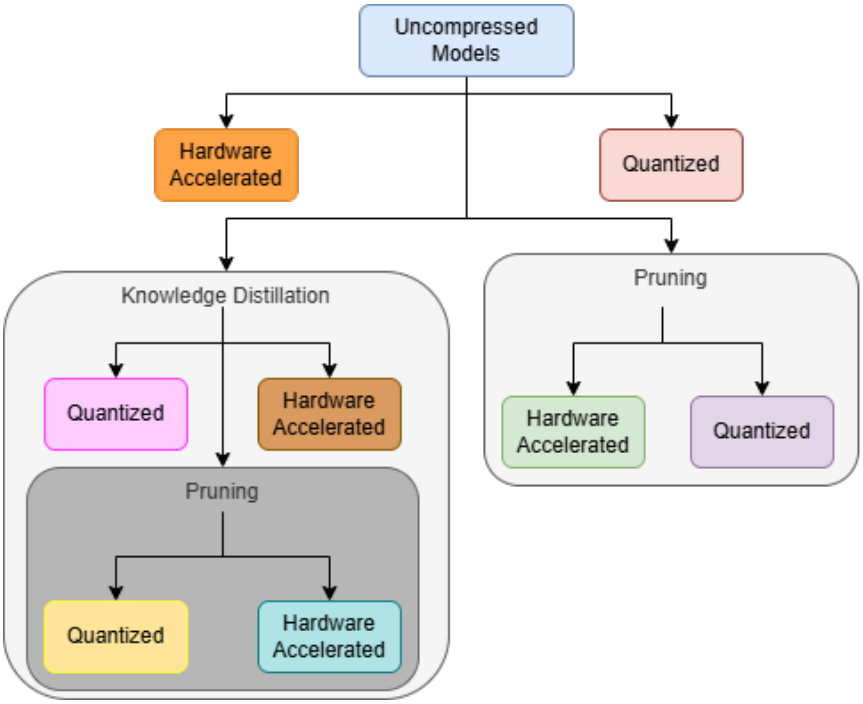


Figure 4: Proposed full-stack system diagram.

In total, 11 compression configurations are applied to the baseline uncompressed models. These comprise the four individual techniques—hardware acceleration, quantisation, knowledge distillation, and pruning—together with seven hybrid configurations formed by combining these methods in different ways.

6.3 Hardware Acceleration

After training in PyTorch, each model checkpoint was exported from its native `.pt` format to a TensorRT `.engine` file in FP32 precision to enable hardware-accelerated inference on the NVIDIA Jetson Orin Nano. This export stage was included so that evaluation could be performed using the deployment runtime intended for the target edge device, rather than relying on the original training framework. While FP32 retains full numerical precision, conversion to TensorRT still enables substantial optimisation through techniques such as graph simplification, layer fusion, and improved kernel scheduling. This made FP32 engine export a suitable deployment format for measuring model performance under realistic edge inference conditions while avoiding changes to the underlying numerical representation.

This approach was used because it reduces inference overhead and improves deployment efficiency on embedded GPU platforms. NVIDIA recommends separating engine building from inference itself: the engine is built and serialized once, then loaded later for repeated inference runs. This avoids repeating the expensive optimization stage each time the application starts and allows TensorRT to exploit hardware-specific optimizations such as reduced-precision execution. A further practical consideration is that .engine files are generally tied to the TensorRT version and hardware platform on which they were created, meaning that engines usually need to be rebuilt if the deployment environment changes significantly.

6.3.1 Quantisation

FP16 and INT8 quantisation was implemented through post-training export of trained PyTorch checkpoints to TensorRT `.engine` files, enabling reduced-precision inference on the NVIDIA Jetson Orin Nano. Rather than modifying the model training stage, quantisation was applied at deployment time in order to reduce computational and memory requirements on the target edge device. All exports were performed at a fixed input resolution of 1280×1280 and with ONNX opset 20 to maintain consistency across models [7, 22]. This allowed quantisation to be evaluated as a practical deployment-stage compression technique for improving inference efficiency while preserving acceptable model performance.

For INT8 engine generation, calibration data was provided during export to estimate representative activation ranges before deployment [22]. Task-specific dataset definitions were used for object detection and pose estimation, with calibration samples drawn from the training split by default and optionally limited by dataset fraction. The resulting TensorRT engines were then evaluated through the same benchmarking and validation pipeline as the other deployed models, allowing their impact on latency, accuracy, and hardware usage to be assessed consistently on the Jetson Orin Nano.

6.3.2 Pruning

Pruning was implemented as structured channel pruning using NVIDIA ModelOpt Fast-NAS [28, 22]. This approach searches for a reduced channel configuration under an explicit FLOP constraint, producing smaller and more efficient network variants while preserving a deployment-compatible architecture. In this work, trained YOLO checkpoints were pruned using target FLOP reductions of 90%, 80%, and 70% where feasible. After this, fine-tune training was applied under the same procedure as the uncompressed models, but for only 100 epochs.

6.3.3 Knowledge Distillation

Knowledge distillation was implemented as a teacher–student training strategy, in which a smaller YOLO student model was optimised using both the standard detection loss and an additional distillation loss. A larger pretrained YOLO teacher model was kept fixed throughout training, with no gradient updates, and both models processed the same input image at each iteration. Rather than distilling only the final predictions, supervision was applied at intermediate feature levels by matching corresponding stages in the teacher and student backbone and neck. Since these paired feature maps did not always have the same channel dimensions, lightweight alignment layers were used to project the student features into the teacher feature space before the distillation loss was computed. This allowed the student to retain its lightweight architecture while benefiting from the stronger internal representations learned by the teacher.

Two feature-distillation methods were evaluated: Channel-Wise Distillation (CWD) and Masked Generative Distillation (MGD) [9, 36]. CWD encourages the student to match the teacher's channel-wise spatial response patterns, helping it learn more discriminative feature representations. MGD instead applies random masking to the student features and trains a small reconstruction module to recover teacher-like representations, promoting stronger semantic understanding and robustness. The overall training objective was defined as

$$\mathcal{L} = \mathcal{L}_{\text{det}} + \lambda_t \mathcal{L}_{\text{distill}},$$

where λ_t followed a decaying schedule, with a larger value early in training and a smaller value later. This placed greater emphasis on teacher guidance during the early stages of representation learning, before gradually shifting focus toward the student's own detection performance.

6.4 Evaluation Metrics and Benchmarking Protocol

Model accuracy was evaluated primarily using $\text{mAP}_{50\text{–}95}$ on a validation subset [6]. $\text{mAP}_{50\text{–}95}$ is a standard object detection metric that measures how well predicted objects match the ground truth in both classification and localisation, describing the detection performance averaged across Intersection over Union (IoU) thresholds from 0.50 to 0.95 in steps of 0.05. This makes it a stricter measure than evaluating at a single threshold. It is expressed mathematically as:

$$\text{mAP}_{50\text{–}95} = \frac{1}{10} \sum_{k=0}^{9} \text{AP}_{0.50+0.05k}$$

where $\text{AP}_{0.50+0.05k}$ denotes the average precision computed at an IoU threshold of $0.50 + 0.05k$. Average precision (AP) is defined as the area under the precision–recall curve, summarising the trade-off between detection precision and recall for a given IoU threshold.

The metric $\text{mAP}_{1\text{–}5}$ was used to analyse pass-detection accuracy, where predictions were matched to ground-truth pass events using temporal tolerances of 1 to 5 seconds rather

than spatial IoU thresholds. This allows pass detection performance to be assessed under increasing levels of timing tolerance. Hardware usage was assessed primarily by GPU utilisation, power draw, and device temperature. These metrics were chosen for their relevance to the NVIDIA Jetson Orin Nano, which operates within an 85°C thermal limit, a 25 W power budget, and a constrained 1024-core CUDA GPU, which provides limited compute resources, especially for multiple heavy computer vision workloads run in parallel [23].

Hardware usage and inference performance were measured for each model by running inference on the same image set over a 600s run. Each run began with five warm-up inferences, which were excluded from analysis. Latency was recorded for every timed inference, while throughput (FPS) was computed as the total number of successful inferences divided by cumulative inference time. In parallel, system telemetry was sampled every 1000ms using NVIDIA's built-in `tegrastats` command, from which mean CPU utilisation, GPU utilisation, RAM usage, temperature, and power draw were computed.

After all models were benchmarked and validated, the efficiency-accuracy trade-off could be analysed. To identify the optimal models in this regard, Pareto analysis was applied in the two-dimensional space, maximising $\text{mAP}_{50\text{–}95}$ while minimising latency [19]. A model i was considered Pareto-dominated and dismissed if there existed another model j such that:

$$\text{mAP}^{(j)}_{50\text{–}95} \geq \text{mAP}^{(i)}_{50\text{–}95} \quad \text{and} \quad \text{LAT}^{(j)} < \text{LAT}^{(i)}$$

The remaining non-dominated models form the Pareto frontier and were used to shortlist best models for full-stack analysis.

6.5 Post-Processing and Higher-level Statistic Generation

The trained and compressed object and keypoint models were then used in a post-processing pipeline to allow for higher-level football analytics to be generated. This stage converted raw model outputs into more complex match statistics: specifically team identity, player possession, team possession, and pass events. To support this, the 2D pitch-coordinate loca-

tions of players and the ball were estimated using a homography computed with RANSAC [13] and temporally stabilised with exponential smoothing. For this projection, the bottom-centre point of each player bounding box, corresponding approximately to the player's feet, and the ball centre were used. Object detections were associated across frames using the ByteTrack MOT algorithm [42].

Team identity was assigned from upper-body player crops with pitch pixels suppressed. The dominant jersey colour was estimated using 2-cluster K-means clustering [45], and the resulting dominant colour was then matched, after temporal smoothing, to manually predefined team colour palettes.

Player possession was estimated from the projected locations of players and the ball, with possession assigned to the nearest player within a predefined distance threshold. Passes were defined as valid same-team transitions in control between different players.

6.6 Full-Stack System Overview

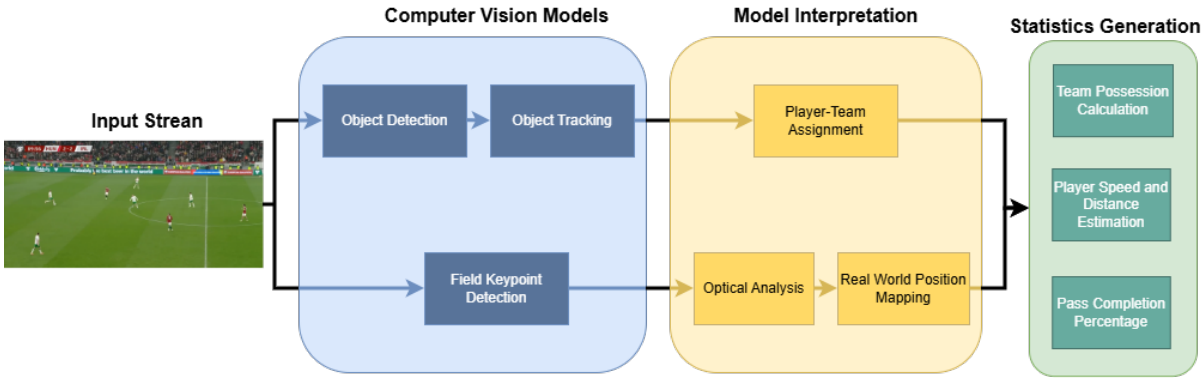


Figure 5: Proposed full-stack system diagram.

Figure 5 illustrates how the trained object detection models, keypoint localisation models, and post-inference processing were integrated into the proposed full-stack system. Each video frame was processed both by an object detector and a pitch keypoint detector, with these intermediate outputs then consumed by the post-inference algorithms described previously. Inference could be performed either on every frame or at a configurable interval, to allow further hardware efficiency customisation. Confidence thresholds were applied to object detection, pitch keypoint detection, team-colour assignment, and pass classification.

7 Results

7.1 Selecting Model Architecture and Dataset

As discussed earlier, preliminary experiments were first carried out to determine the model family and training dataset to be used for the majority of the study. These experiments compared candidate object detection architectures in terms of analytical performance and latency, and also compared several labelled football broadcast datasets.

Figure 6: Architecture and dataset comparison.

(a) Architectures. (b) Datasets.

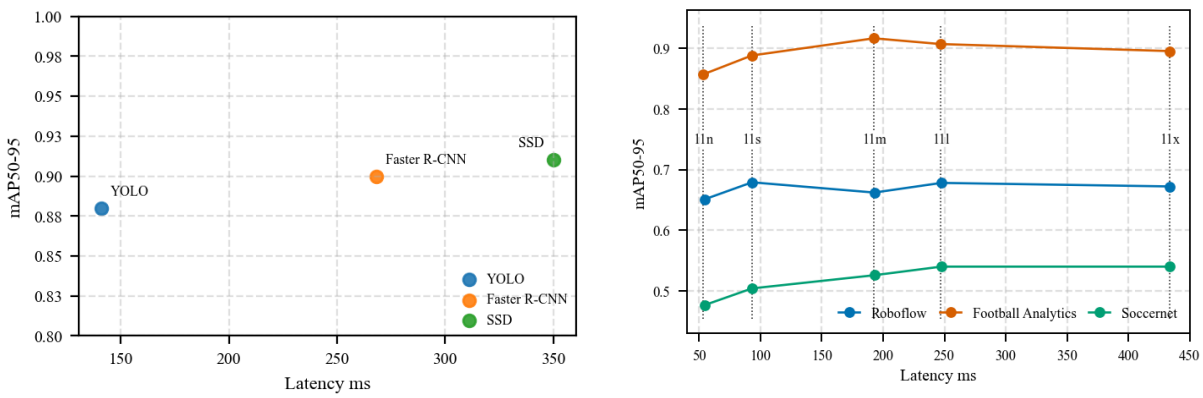


Figure 6a compares the analytical performance and latency of a range of object detection model families. Although Faster R-CNN [43] and SSD achieved slightly higher initial accuracy, the YOLO family [37] provided substantially lower latency on the Jetson Orin Nano Dev Kit. As a result, YOLO-based models were selected for the subsequent object detection experiments, as they offered the most suitable balance between accuracy and real-time edge deployment performance.

Figure 6b shows the preliminary dataset comparison carried out using the SoccerNet [3], Roboflow [31], and Football Analytics [33] datasets. Among these, the Football Analytics dataset achieved the strongest initial mAP_{50-95}, at approximately 91%, and was therefore selected for the majority of the object detection study.

7.2 Summary of Model Training

With the model family and dataset selected, the full training and compression study was then carried out. In total, 488 unique model artefacts were generated and evaluated across the object detection and pitch keypoint tasks. This included 104 **YOLO11** models, 152 **YOLO11-pose** models, 72 **YOLO26** models, and 160 **YOLO26-pose** models. Table 5 summarises the final model inventory used throughout the study.

Table 5: Model inventory.

Category	**Count**
Total Models	488
Object Models	212
Pitch Models	276
Uncompressed	40
Accelerated	40
Distilled	70
Pruned	68
Quantized	40
Combinations	270

As shown in Table 5, the majority of the model inventory consisted of compressed variants rather than baseline models, reflecting the central aim of the study: to examine how different deployment-focused compression strategies affect performance on edge hardware.

7.3 Compression Analysis

This section analyses the effect of the main optimisation and compression strategies explored in this work, namely hardware acceleration, quantisation, pruning, knowledge distillation, and combinations of these techniques. For each approach, the change in mAP50-95 and latency is examined in order to assess how deployment efficiency can be improved while maintaining acceptable model accuracy. Together, these subsections provide a structured comparison of the main accuracy–efficiency trade-offs, and help identify the compression pathways that are most suitable for real-time embedded soccer analysis.

Tables 6 and 8 condense the benchmark results into task-specific averages for the base techniques, while Tables 7 and 9 isolate the combined compression paths. Each value is

measured relative to the matching uncompressed student baseline for the same family and scale, which makes the individual techniques and the multi-stage combinations directly comparable at a summary level.

Table 6: Object base compression summary.

Technique	**Δ mAP$_{50–95}$**	**Lat. ↓**
Hardware acceleration	-0.3%	31.3%
Quantisation	-4.9%	56.6%
FP16	-1.6%	53.8%
INT8	-8.2%	59.4%
Pruning	-2.3%	15.4%
90% pruning	-1.6%	13.5%
80% pruning	-2.4%	15.6%
70% pruning	-4.6%	22.0%
Distillation	-1.3%	0.1%
Small jump	-1.1%	-0.5%
Medium jump	-0.7%	0.0%
Large jump	-2.1%	0.9%

Note: Distillation jump size is grouped by teacher-size gap: small = next size up, medium = two sizes up, and large = three or more sizes up.

Table 7: Object combined compression summary.

Combination	**Δ mAP$_{50–95}$**	**Lat. ↓**
Distilled	-4.4%	36.1%
Distilled + Accelerated	-1.5%	23.4%
Distilled + FP16	-2.8%	42.0%
Distilled + INT8	-8.9%	42.9%
Pruned	-5.8%	55.9%
Pruned + Accelerated	-2.8%	40.5%
Pruned + FP16	-4.4%	61.6%
Pruned + INT8	-10.1%	65.6%
Distilled + Pruned	-3.9%	26.6%
Distilled + Pruned	-1.6%	8.3%
Distilled + Pruned + Accel.	-2.2%	23.1%
Distilled + Pruned + FP16	-3.3%	37.7%
Distilled + Pruned + INT8	-8.6%	37.3%

Note: Rows ending with "Accelerated" or "Accel." refer to TensorRT engine deployment without FP16 or INT8 quantisation. FP16 and INT8 rows are also accelerated deployments, but include reduced-precision quantisation as an additional step.

Table 8: Pose base compression summary.

Technique	Δ $mAP_{50–95}$	Lat. ↓
Hardware acceleration	-0.2%	29.9%
Quantisation	-6.5%	59.6%
FP16	-1.3%	56.0%
INT8	-11.8%	63.2%
Pruning	-24.6%	16.3%
90% pruning	-23.7%	15.2%
80% pruning	-24.5%	16.9%
70% pruning	-26.3%	17.3%
Distillation	-4.7%	0.6%
Small jump	-4.8%	0.2%
Medium jump	-6.3%	0.6%
Large jump	-3.0%	1.2%

Note: Distillation jump size is grouped by teacher-size gap: small = next size up, medium = two sizes up, and large = three or more sizes up.

Table 9: Pose combined compression summary.

Combination	Δ $mAP_{50–95}$	Lat. ↓
Distilled	-12.6%	39.7%
Distilled + Accelerated	-4.8%	24.3%
Distilled + FP16	-4.7%	44.9%
Distilled + INT8	-28.2%	49.9%
Pruned	-30.8%	50.5%
Pruned + Accelerated	-24.5%	39.1%
Pruned + FP16	-24.5%	54.4%
Pruned + INT8	-43.3%	58.1%
Distilled + Pruned	-9.7%	33.6%
Distilled + Pruned	-3.4%	11.3%
Distilled + Pruned + Accel.	-3.6%	33.0%
Distilled + Pruned + FP16	-3.7%	43.5%
Distilled + Pruned + INT8	-28.2%	46.7%

Note: Rows ending with "Accelerated" or "Accel." refer to TensorRT engine deployment without FP16 or INT8 quantisation. FP16 and INT8 rows are also accelerated deployments, but include reduced-precision quantisation as an additional step.

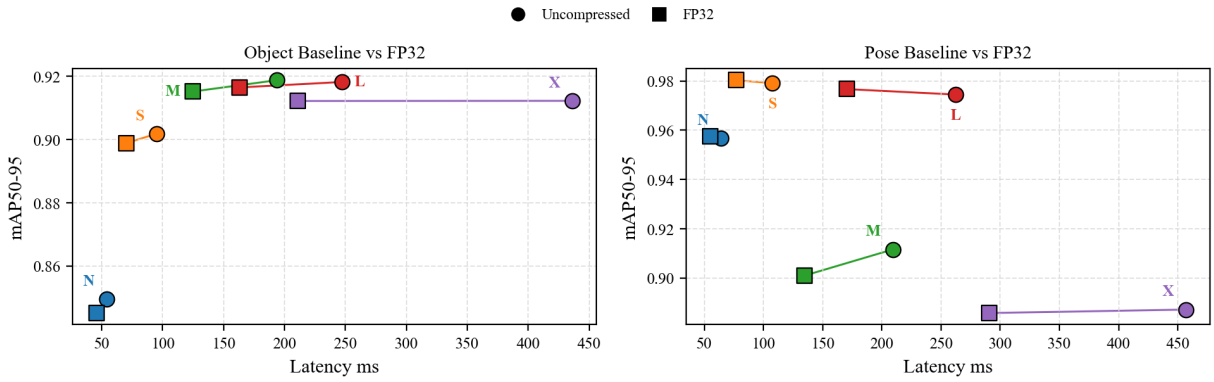
At this higher-level view, the object models benefit most strongly from quantisation in the base-technique table and from pruning–quantisation in the combinations table, since these options produce the largest latency reductions while keeping the mean $mAP_{50–95}$ drop within a moderate range. The pose models are much more sensitive, especially under pruning and INT8-based combinations, where the efficiency gains remain large but the average $mAP_{50–95}$ loss becomes substantially higher. Distillation on its own changes latency

very little in either task, so its main role is not as a standalone acceleration method, but as a supporting step inside broader compression paths.

7.3.1 Hardware Acceleration

Figure 7 presents the average mAP50-95 versus latency relationship for the uncompressed and FP32-accelerated YOLO object and pose model families. By averaging the YOLO11 and YOLO26 variants at each model scale, the figure provides a higher-level view of how deployment through TensorRT-based hardware acceleration affects the overall accuracy–latency trade-off. This allows the effect of acceleration to be separated from architectural differences between individual models, making the general performance shift across scale easier to interpret.

Figure 7: Hardware acceleration trade-off.

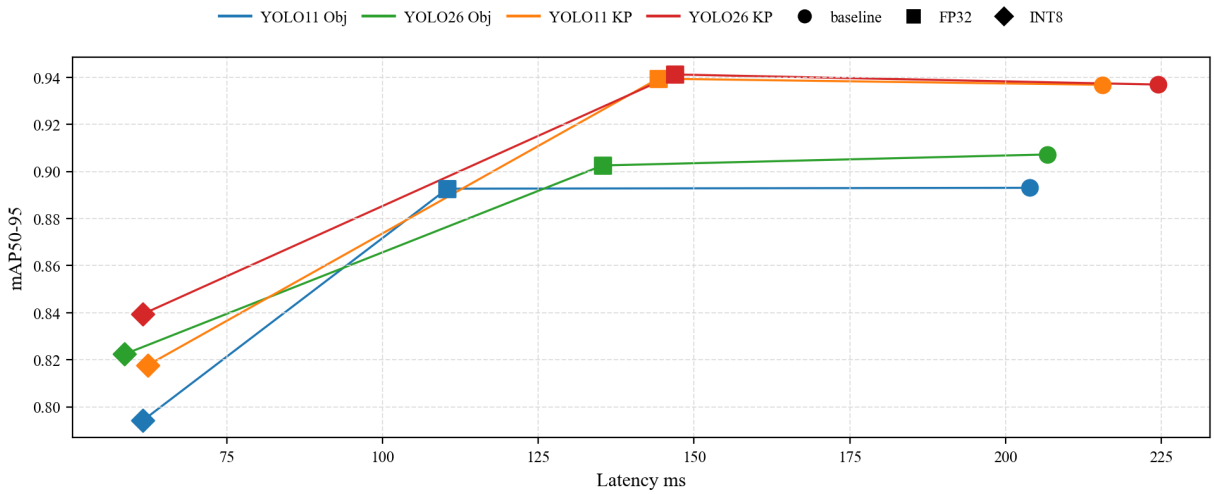


The results show that hardware acceleration provides a substantial reduction in latency across all model scales while preserving almost identical mAP50-95 performance. In each family-task group, the FP32-accelerated points move consistently leftward relative to their uncompressed counterparts, indicating faster inference with negligible accuracy loss. This suggests that deployment optimisation alone provides a strong baseline improvement before more aggressive compression techniques are considered. It is also notable that the relative ordering of the models is preserved after acceleration, meaning that larger models remain more accurate but slower, while smaller models remain faster but less accurate. Overall, hardware acceleration improves deployability without materially altering the underlying accuracy hierarchy.

7.3.2 Quantisation

Figure 8 shows the average mAP50-95 versus latency trade-off for baseline, FP32, and INT8 deployment stages across the YOLO11 and YOLO26 object and pose families. Each line traces the progression of a family-task group through increasingly aggressive deployment optimisation, allowing the effect of quantisation on both inference speed and accuracy retention to be examined in a structured way. This makes it possible to compare whether object and pose models respond similarly to reduced numerical precision.

Figure 8: Quantisation trade-off.

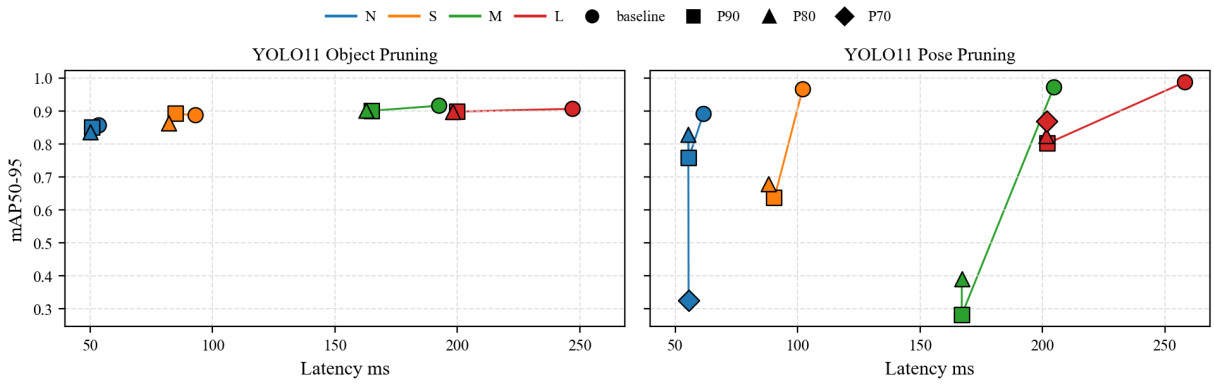


The figure shows that quantisation produces clear latency improvements, with INT8 giving the largest speedup in every family-task group. However, the associated accuracy cost is much more pronounced than in the hardware acceleration case. FP32 acceleration preserves accuracy almost exactly, whereas the move to INT8 shifts the points further left but also noticeably downward. This effect is particularly clear for the object models, where latency is reduced substantially but mAP50-95 falls by a visible margin. The pose models also experience a drop, though they remain at relatively high absolute accuracy. Overall, the results indicate that quantisation is effective for improving runtime efficiency, but that the gain is achieved through a more explicit accuracy–latency trade-off than hardware acceleration alone.

7.3.3 Pruning

Figure 9 illustrates the mAP50-95 versus latency relationship for YOLO11 object and pose models under increasing pruning strength. Separate N, S, M, and L lines are shown for the baseline, P90, P80, and P70 checkpoints, allowing the effect of structured pruning to be examined both across pruning intensity and across model scale. This presentation makes it possible to assess whether pruning shifts the models toward a more favourable operating point or instead introduces excessive accuracy degradation.

Figure 9: Pruning trade-off.

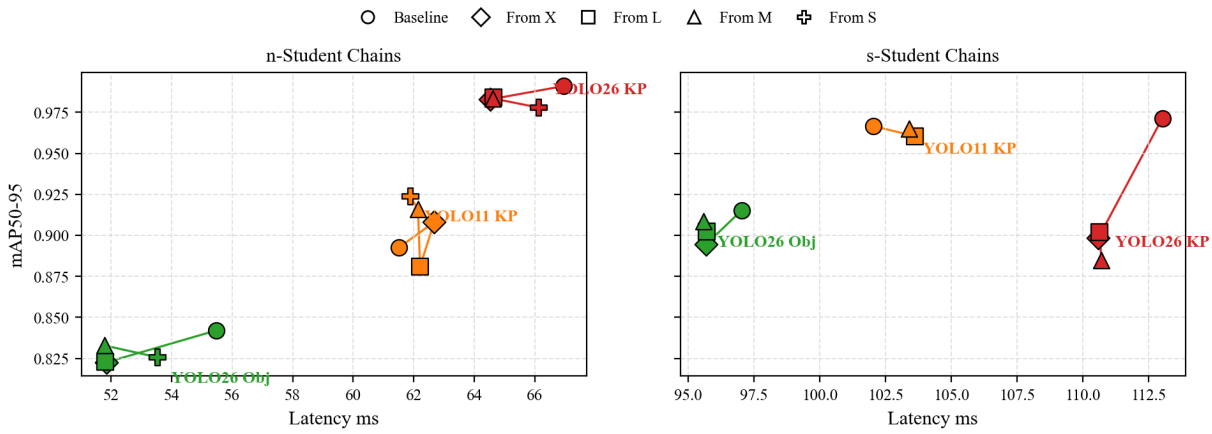


The object detection results show that pruning generally reduces latency while causing only moderate reductions in mAP50-95, particularly for the larger model scales. In several cases, the pruned object models remain close to the baseline in accuracy while offering meaningful speed improvements, which suggests that object detection models retain a degree of redundancy that pruning can exploit. In contrast, the pose models are much more sensitive to pruning. While mild pruning can still provide a useful trade-off, heavier pruning leads to severe reductions in mAP50-95 for several scales, especially at the strongest pruning settings. This indicates that the keypoint models are less robust to parameter removal and reach an accuracy floor much more quickly. Overall, the figure suggests that pruning is a more reliable compression method for object detection than for pitch keypoint estimation.

7.3.4 Knowledge Distillation

Figure 10 presents the effect of knowledge distillation on YOLO11s object and pose models by comparing the baseline student against students distilled from larger teacher models. The figure shows whether transferring knowledge from progressively larger teachers produces measurable improvements in the student accuracy–latency trade-off, while keeping the underlying student architecture fixed. This makes it possible to isolate the contribution of distillation itself rather than conflating it with changes in model size.

Figure 10: Knowledge distillation trade-off.

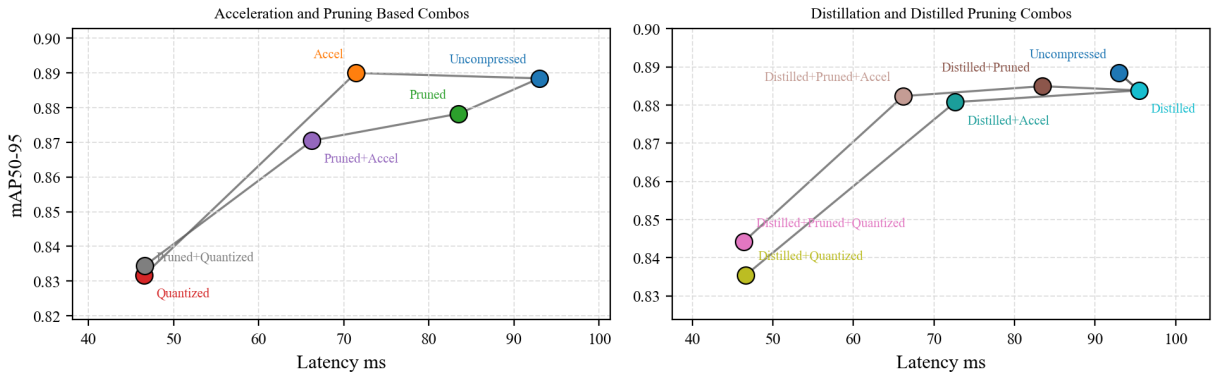


The results show that distillation produces modest but consistent accuracy improvements while leaving latency almost unchanged. For the object student, the distilled variants cluster very closely in latency yet slightly outperform the baseline in mAP50-95, indicating that the student benefits from teacher guidance without incurring a meaningful runtime cost. A similar pattern is visible for the keypoint student, where the distilled models remain close to the baseline latency while achieving comparable or slightly improved accuracy. The gains are not dramatic, but they are important because they move the student upward rather than forcing the usual downward shift associated with compression. Overall, the figure suggests that knowledge distillation is an accuracy-preserving compression support technique rather than a direct latency-reduction method, and is most valuable when used to strengthen smaller deployable models.

7.3.5 Combinations of Different Techniques

Figure 11 brings the different compression strategies together by tracing the mAP50-95 versus latency paths of YOLO11s object models under acceleration, pruning, quantisation, distillation, and their combined variants. Rather than treating each technique in isolation, the figure shows how successive optimisation steps reshape the overall accuracy–latency frontier. This provides a compact way to compare which sequences of transformations produce the most favourable deployment trade-offs.

Figure 11: Combined compression trade-off.



The combined results show that no single path dominates in all respects, and that the best choice depends on whether latency reduction or accuracy retention is prioritised. Acceleration alone provides a strong first improvement, while quantisation gives the largest latency reduction but also the largest drop in mAP50-95. Pruning tends to occupy the middle ground, offering useful speed gains with smaller accuracy loss than quantisation. Distillation-based paths are especially notable because they help preserve or recover accuracy, making the combined distilled-pruned or distilled-accelerated variants more balanced overall. The figure therefore reinforces the broader pattern seen throughout this section: the most effective deployment candidates are usually not the most aggressively compressed models, but the intermediate configurations that balance accuracy preservation with meaningful runtime improvement.

7.4 Hardware Analysis

7.4.1 Latency

Table 10 summarises how each compression stage changes end-to-end inference latency across the object and pose models. To keep the comparison compact, the values average the paired YOLO11 and YOLO26 results within each size class, while still separating the object and pose tasks. This makes it easier to see whether the same optimisation strategy produces similar runtime benefits across both detection and keypoint estimation.

Table 10: Average latency reduction by stage.

	Object					**Pose**				
Stage	**N**	**S**	**M**	**L**	**X**	**N**	**S**	**M**	**L**	**X**
Uncompressed	54.37	95.00	193.81	247.34	436.18	64.23	107.53	209.28	262.16	456.87
Accelerated	15.7%	26.1%	35.7%	34.1%	51.8%	14.3%	28.2%	35.7%	35.1%	36.4%
Distilled	15.8%	25.6%	36.4%	34.6%	–	15.5%	28.7%	36.3%	36.2%	–
Pruned	13.0%	31.8%	43.8%	46.8%	–	27.5%	33.8%	46.1%	47.9%	–
Quantized	21.1%	52.5%	71.8%	74.2%	78.5%	32.7%	57.1%	72.3%	75.0%	78.7%

The latency results show that quantisation delivers the largest and most consistent speed-up for both tasks, with reductions rising from around one fifth at the smallest object models to almost 79% for the X variants. Acceleration also produces a strong baseline improvement, particularly for the larger models where the absolute runtime cost is highest, while pruning offers a more moderate middle ground on the sizes where paired runs are available. Distillation improves latency only modestly by comparison, which reinforces the earlier observation that its main contribution is preserving accuracy rather than directly driving runtime reductions.

7.4.2 GPU Usage

Table 11 reports the corresponding change in GPU utilisation after each compression stage. Because the same averaging scheme is used here, the table allows the hardware load of object and pose inference to be compared directly across model scale, while also showing whether reductions in latency are accompanied by lower pressure on the GPU.

Table 11: Average GPU reduction by stage.

	Object					Pose				
Stage	**N**	**S**	**M**	**L**	**X**	**N**	**S**	**M**	**L**	**X**
Uncompressed	63.23	76.81	87.44	89.50	92.61	67.03	78.76	87.71	91.62	93.04
Accelerated	17.7%	21.2%	13.9%	10.2%	5.1%	17.7%	19.8%	14.2%	11.9%	9.8%
Distilled	8.5%	10.9%	6.4%	5.0%	–	19.8%	20.3%	12.6%	11.9%	–
Pruned	14.5%	12.1%	11.3%	7.6%	–	23.9%	23.4%	17.0%	17.3%	–
Quantized	20.8%	32.5%	33.9%	33.5%	24.1%	25.3%	40.7%	36.7%	33.6%	25.9%

The GPU results broadly follow the latency trends, but with smaller absolute gains for most techniques. Quantisation again gives the largest reductions, especially for the S, M, and L variants, while hardware acceleration offers reliable but more moderate savings. Distillation and pruning reduce GPU usage in several cases, but the improvements are clearly less dramatic than those seen for latency, suggesting that these techniques reshape the compute profile only partially rather than fundamentally lowering accelerator demand.

7.4.3 CPU Usage

Table 12 shifts the focus from accelerator load to host-side overhead. This comparison is particularly useful because some optimisations that improve inference speed on the GPU can still increase CPU involvement through scheduling, memory movement, or engine management, so the table helps expose trade-offs that are not visible from latency alone.

Table 12: Average CPU reduction by stage.

	Object					Pose				
Stage	**N**	**S**	**M**	**L**	**X**	**N**	**S**	**M**	**L**	**X**
Uncompressed	13.34	8.69	6.16	5.62	5.46	12.49	8.39	5.91	6.44	6.01
Accelerated	29.0%	14.1%	22.3%	26.2%	51.6%	28.6%	14.1%	22.2%	38.3%	59.5%
Distilled	14.3%	7.3%	11.8%	10.4%	–	28.8%	14.2%	23.1%	38.5%	–
Pruned	10.6%	0.9%	0.5%	-1.2%	–	21.5%	8.0%	5.1%	23.7%	–
Quantized	25.8%	-9.8%	-34.5%	-36.2%	-12.7%	20.6%	-18.6%	-44.8%	-22.8%	-0.3%

The CPU results are more mixed than the GPU results. Hardware acceleration gives the clearest CPU benefit overall, especially for the X models, whereas pruning yields only small

gains and in some object cases is close to neutral. Most notably, quantisation frequently increases CPU usage for the medium, large, and extra-large models, producing negative reductions, which suggests that part of the optimisation cost is being shifted away from the accelerator and onto the CPU. Distillation sits between these extremes, with moderate CPU savings for the available N–L sizes but no evidence that it is a dominant hardware-efficiency technique by itself.

7.4.4 Power Draw

Table 13 shows how the same transformations affect total power draw. Since deployment on embedded hardware is constrained not only by raw throughput but also by energy availability and thermal headroom, these measurements provide a more deployment-oriented view of which techniques meaningfully reduce system-wide operating cost.

Table 13: Average power reduction by stage.

	Object					**Pose**				
Stage	**N**	**S**	**M**	**L**	**X**	**N**	**S**	**M**	**L**	**X**
Uncompressed	12.77	14.99	16.53	17.00	17.62	13.30	15.07	16.52	17.16	17.65
Accelerated	18.2%	10.9%	3.5%	3.6%	-1.2%	45.9%	31.9%	17.5%	16.9%	-0.7%
Distilled	19.1%	13.0%	2.5%	3.8%	–	-0.7%	-0.7%	0.3%	0.0%	–
Pruned	20.7%	15.6%	6.1%	6.5%	–	2.8%	1.2%	1.2%	2.1%	–
Quantized	53.1%	53.1%	45.4%	36.4%	23.7%	52.2%	50.8%	44.0%	34.8%	24.0%

The power results indicate that quantisation is the strongest method for reducing energy demand, with savings of roughly 24% to 53% across the reported sizes. Hardware acceleration also lowers power substantially for the smaller pose models, but its effect weakens for larger object models and is almost neutral for the X variants. By contrast, distillation and pruning have relatively small influence on power draw, which implies that their main value lies elsewhere, such as accuracy retention or balanced latency reduction, rather than direct energy savings.

7.4.5 Temperature

Table 14 extends the hardware analysis to thermal behaviour. Temperature is an important deployment metric because sustained improvements in latency or power are only truly useful if they also reduce heat generation enough to avoid thermal throttling and maintain stable operation over long runs.

Table 14: Average temperature reduction by stage.

	Object					**Pose**				
Stage	**N**	**S**	**M**	**L**	**X**	**N**	**S**	**M**	**L**	**X**
Uncompressed	58.79	60.43	63.14	63.09	64.64	58.16	60.97	63.16	64.14	64.76
Accelerated	5.7%	1.6%	-0.3%	-1.1%	-1.5%	15.1%	11.0%	6.1%	5.6%	-0.7%
Distilled	6.8%	1.2%	-0.5%	-1.2%	–	-0.6%	-0.3%	1.5%	0.2%	–
Pruned	6.8%	3.1%	1.3%	-0.6%	–	0.6%	1.1%	1.5%	1.1%	–
Quantized	19.1%	16.2%	13.7%	9.8%	6.7%	16.4%	18.0%	15.3%	11.9%	8.3%

The temperature results again favour quantisation, which produces the largest and most uniform reductions across both object and pose models. Hardware acceleration helps noticeably for the smaller and mid-sized pose models, but has only limited or even slightly negative impact for the larger object variants, showing that faster execution does not automatically translate into cooler operation. Distillation and pruning have the weakest and least consistent thermal effects, so while they remain useful from an accuracy–efficiency perspective, they are not the primary tools for improving thermal behaviour.

7.5 Full-Stack System

7.5.1 Full-Stack System Performance

Table 15 summarises the five strongest full-stack model combinations under the rank-02 replay setting. To keep the comparison focused on deployment-relevant trade-offs, the table reports pass-detection accuracy, dominant-team possession agreement, runtime throughput, and GPU usage.

Table 15: Top five full-stack combinations.

Object Model	Keypoint Model	Pass mAP_{1-5}	ΔPos vs. GT	FPS	GPU (%)
26m (80%, FP16)	**26n (80%, FP16)**	**0.697**	**12.22%**	**6.51**	**39.61**
26l (FP16)	26n (FP16)	0.687	1.11%	5.57	51.50
26l (FP16)	26n (KD-26m, 80%, INT8)	0.655	7.71%	5.75	53.39
26m (80%, FP16)	26n (KD-26x, 80%, FP16)	0.583	10.83%	6.49	44.27
26s (Uncompressed)	26n (KD-26m, 80%, INT8)	0.574	7.54%	6.18	47.64

Note: Model labels use the format *Architecture (Compression Settings)*. Here, 80% denotes 80% structured pruning, KD-26m and KD-26x denote knowledge distillation from 26m and 26x teacher models respectively, and FP16 and INT8 denote quantisation. ΔPos vs. GT gives the absolute difference in dominant-team possession share relative to ground truth.

Table 16 provides the corresponding average hardware profile across all 16 evaluated combinations in the same rank-02 replay batch.

Table 16: Average full-stack hardware profile.

Summary	Latency (ms)	FPS	Power (W)	Temp (°C)	GPU (%)
Mean across 16 combos	155.89	6.36	3.83	52.34	47.35

8 Impact of Research

With recent scrutiny of Artificial Intelligence within politics and public eye, it is also important to discuss the ethical considerations made in the preparation of this project, as well as the efforts placed towards sustainability.

8.1 Ethical Considerations

The first key ethical consideration is the use of individuals in model datasets. The two primary object detection datasets, sourced by *Soccer-Net* and *Roboflow*, are compiled entirely with video and images from the 5 major European soccer leagues. These leagues, and by extension the players involved, allow fair use of all data, as long as it is not used for commercial use, which this project will not be for. This allows these datasets to be ethi-

cally used. As a further step in reducing these privacy concerns, the proposed system does not perform any form of personal identification, facial recognition, or biometric analysis. Players are represented solely through abstract bounding boxes and keypoints to ensure that analysis remains focused on game statistics, rather than personal identity.

8.2 Sustainability Considerations

Deep learning models are often associated with high energy consumption, particularly when trained or deployed in cloud-based environments without consideration for efficiency [4]. This project directly addresses these concerns by focusing on computational efficiency and edge deployment, aiming to deliver accurate analytics while minimizing energy usage through techniques such as quantization, input resolution scaling, pruning, and knowledge distillation. Deploying inference on an edge device such as the Jetson Orin Nano also reduces the need for data transmission to centralised servers, lowering the energy footprint associated with networking and cloud computation, and aligning with principles of sustainable artificial intelligence.

The last remaining sustainability concern is the required energy cost of repeated model training during this project's experimentation. This will be mitigated by limiting the number of model configurations explored, avoiding redundant training runs, and prioritising model reuse and controlled benchmarking to ensure energy-aware evaluation.

9 Proposed Future Work

Even with the current progress made, future plans can be made to ensure the project reaches a strong conclusion. This can be separated into end-to-end system functionality improvements, further system optimization analysis, and further model compression analysis.

9.1 System Functionality Improvements

Proposed system functionality improvements include adding new soccer-related statistics, and introducing/modifying additional web-app hyper-parameters. One high-level statistic that will be explored is pass completion percentages, displaying how often a team's passes are successful. This can be determined by detecting when ball possession moves from one player to another within the same team within a certain period of time. These pieces of information are available within the system already, and so all that is needed is the implementation.

The system's architecture itself will also be improved. Multiple hyper-parameters should be configurable by the user, including the Intersection over Union (IoU) threshold, an object detection frequency rate, a team assignment interval, and possession radius thresholds. Inference-time model selection will be implemented, allowing users to choose which model architecture to use.

9.2 System Optimization

Future system optimization will primarily target end-to-end latency reduction, with emphasis on both inference and post-inference stages. Object detection interval reduction will be further explored across multiple trained models to quantify the latency/accuracy trade-off. The interval between updating a player's team will also be investigated, as benchmarking indicates that per-frame execution is costly and unnecessary. In addition, replacing the key-point detection model with a classical approach [24] will be explored to reduce inference overhead. Finally, asynchronous processing between inference and post-processing stages will be examined, to assess potential parallelization benefits that utilise both the CPU and GPU.

9.3 Model Compression

Further model compression analysis will extend the preliminary input-resolution scaling work done, by benchmarking it across different trained models. This will include evaluating

models that are trained at reduced resolutions, but inferred at higher resolutions to assess robustness to resolution mismatch. Quantization strategies, including post-training quantization and quantization-aware training, will be compared in terms of inference latency, power consumption, and analytical performance on the Jetson Nano. Finally, pruning-based approaches (both structured and unstructured) and knowledge distillation techniques will be investigated to quantify the trade-offs between model complexity, inference speed, and analytical accuracy.

References

[1] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016. doi: 10.1109/ICIP.2016.7533003.

[2] Yen-Jui Chu, Jheng-Wei Su, Kai-Wen Hsiao, Chi-Yu Lien, Shu-Ho Fan, Min-Chun Hu, Ruen-Rone Lee, Chih-Yuan Yao, and Hung-Kuo Chu. Sports field registration via keypoints-aware label condition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3523–3530, June 2022.

[3] Anthony Cioppa, Silvio Giancola, Adrien Deliege, Le Kang, Xin Zhou, Zhiyu Cheng, Bernard Ghanem, and Marc Van Droogenbroeck. Soccernet-tracking: Multiple object tracking dataset and benchmark in soccer videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3491–3502, 2022.

[4] Radosvet Desislavov, Fernando Martínez-Plumed, and José Hernández-Orallo. Trends in ai inference energy consumption: Beyond the performance-vs-parameter laws of deep learning. *Sustainable Computing: Informatics and Systems*, 38:100857, 2023.

[5] Elan Dubrofsky. Homography estimation. *Diplomová práce. Vancouver: Univerzita Britské Kolumbie*, 5, 2009.

[6] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010. ISSN 1573-1405. doi: 10.1007/s11263-009-0275-4. URL https://doi.org/10.1007/s11263-009-0275-4.

[7] Amir Gholami, Sehoon Kim, Dong Zhen, Zhewei Yao, Michael Mahoney, and Kurt Keutzer. *A Survey of Quantization Methods for Efficient Neural Network Inference*, pages 291–326. 01 2022. ISBN 9781003162810. doi: 10.1201/9781003162810-13.

[8] Shinjit Ghosh, Shaili Verma, and S Deepa. Sports analysis: Analyzing tennis using deep learning and keypoint extraction. In *2024 International Conference on Electronic*

Systems and Intelligent Computing (ICESIC), pages 151–156, 2024. doi: 10.1109/ICESIC61777.2024.10846339.

[9] Jianping Gou, Baosheng Yu, Stephen J. Maybank, and Dacheng Tao. Knowledge distillation: A survey. *International Journal of Computer Vision*, 129(6):1789–1819, June 2021. ISSN 1573-1405. doi: 10.1007/s11263-021-01453-z. URL https://doi.org/10.1007/s11263-021-01453-z.

[10] Hmzbo. Football-analytics-with-deep-learning-and-computer-vision. https://github.com/Hmzbo/Football-Analytics-with-Deep-Learning-and-Computer-Vision.

[11] Future Market Insights. Sports analytics market size, trends & forecast 2025 to 2035. Industry report, 2025. URL https://www.futuremarketinsights.com/reports/sports-analytics-market. Sports analytics market growth driven by increasing digitalization and data-driven decision tools.

[12] Michael Lewis. *Moneyball: The Art of Winning an Unfair Game*. W. W. Norton & Company, 2003.

[13] Xiaowei Li, Yue Liu, Yongtian Wang, and Dayuan Yan. Computing homography with RANSAC algorithm: a novel method of registration. In Chung-Sheng Li and Minerva M. Yeung, editors, *Electronic Imaging and Multimedia Technology IV*, volume 5637, pages 109 – 112. International Society for Optics and Photonics, SPIE, 2005. doi: 10.1117/12.579121. URL https://doi.org/10.1117/12.579121.

[14] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft coco: Common objects in context, 2015. URL https://arxiv.org/abs/1405.0312.

[15] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.

[16] Wei-Lwun Lu, Jo-Anne Ting, James J. Little, and Kevin P. Murphy. Learning to track and identify players from broadcast sports videos. *IEEE Transactions on Pattern*

Analysis and Machine Intelligence, 35(7):1704–1716, 2013. doi: 10.1109/TPAMI.2012.242.

[17] Debapriya Maji, Soyeb Nagori, Manu Mathew, and Deepak Poddar. Yolo-pose: Enhancing yolo for multi person pose estimation using object keypoint similarity loss. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 2637–2646, June 2022.

[18] Peter Sunny Shanthveer Markappa, Christian O'Leary, and Conor Lynch. A review of yolo models for soccer-based object detection. In *2024 Sixth International Conference on Intelligent Computing in Data Sciences (ICDS)*, pages 1–7, 2024. doi: 10.1109/ICDS62089.2024.10756443.

[19] Christopher A. Mattson and Achille Messac. Pareto frontier based concept selection under uncertainty, with visualization. *Optimization and Engineering*, 6(1):85–115, 2005. ISSN 1573-2924. doi: 10.1023/B:OPTE.0000048538.35456.45. URL https://doi.org/10.1023/B:OPTE.0000048538.35456.45.

[20] Thomas Mooney, Peter Corcoran, and Noel O'Connor. Computer vision for sports: Current applications and research topics. *Computer Vision and Image Understanding*, 124:18–35, 2014. doi: 10.1016/j.cviu.2014.04.011.

[21] mradovic38. football_analysis: Football analysis repository. https://github.com/mradovic38/football_analysis.

[22] NVIDIA. TensorRT Model Optimizer. https://developer.nvidia.com/tensorrt, 2026.

[23] NVIDIA Corporation. Jetson orin nano developer kit, 2024. URL https://developer.nvidia.com/embedded/jetson-orin-nano-devkit.

[24] Kenji Okuma, James J Little, and David G Lowe. Automatic rectification of long image sequences. In *Asian conference on computer vision*, volume 9, page 7, 2004.

[25] Daniil Osokin. Real-time 2d multi-person pose estimation on cpu: Lightweight openpose. In *arXiv preprint arXiv:1811.12004*, 2018.

[26] PricewaterhouseCoopers. Streaming the game: How the rise of digital platforms is changing sports consumption. PwC Industry Report, 2025. URL https://www.pwc.com/us/en/industries/tmt/library/sports-streaming-platforms.html. Retrieved January 2026, from https://www.pwc.com/us/en/industries/tmt/library/sports-streaming-platforms.html.

[27] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

[28] R. Reed. Pruning algorithms-a survey. *IEEE Transactions on Neural Networks*, 4(5): 740–747, 1993. doi: 10.1109/72.248452.

[29] Ralf Rein and Daniel Memmert. Big data and tactical analysis in elite soccer: future challenges and opportunities for sports science. *SpringerPlus*, 5(1):1410, 2016. doi: 10.1186/s40064-016-3108-2.

[30] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6):1137–1149, 2017. doi: 10.1109/TPAMI.2016.2577031.

[31] Roboflow. sports: Sports computer vision datasets and tools. https://github.com/roboflow/sports.

[32] Roboflow. Football players detection dataset, 2024. URL https://universe.roboflow.com/roboflow-jvuqo/football-players-detection-3zvbc/dataset/1. Roboflow Universe dataset.

[33] Roboflow Universe. Football Analytics (gnykk) — Model 1. https://universe.roboflow.com/football-analytics-o89pg/football-analytics-gnykk/model/1, .

[34] Roboflow Universe. Football Field Detection (f07vi) — Dataset, Version 15. https:

//universe.roboflow.com/roboflow-jvuqo/football-field-detection-f07vi/dataset/15, .

[35] Stats Perform. Stats Perform Extends Long-Term Official Statistics Partnership for Premier League, EFL and SPFL. *Stats Perform Press Releases*, August 2023.

[36] Juan Terven, Diana-Margarita Córdova-Esparza, and Julio-Alejandro Romero-González. A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas. *Machine Learning and Knowledge Extraction*, 5 (4):1680–1716, 2023. ISSN 2504-4990. doi: 10.3390/make5040083. URL https://www.mdpi.com/2504-4990/5/4/83.

[37] Ultralytics. Ultralytics yolo, 2024. URL https://github.com/ultralytics/ultralytics. YOLOv8 and YOLOv11 object detection and pose estimation framework.

[38] Ultralytics. Ultralytics configuration documentation. https://docs.ultralytics.com/usage/cfg/, 2026.

[39] Jingdong Wang, Ke Sun, Tianheng Cheng, Borui Jiang, Chaorui Deng, Yang Zhao, Dong Liu, Yadong Mu, Mingkui Tan, Xinggang Wang, Wenyu Liu, and Bin Xiao. Deep high-resolution representation learning for visual recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(10):3349–3364, 2021. doi: 10.1109/TPAMI.2020.2983686.

[40] Haolin Yang, Jiayuan Rao, Haoning Wu, and Weidi Xie. Soccermaster: A vision foundation model for soccer understanding. *arXiv preprint arXiv:2512.11016*, 2025.

[41] Lu Yu and Wei Xiang. X-pruner: explainable pruning for vision transformers. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 24355–24363, 2023. doi: 10.1109/CVPR52729.2023.02333.

[42] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box. 2022.

[43] Hengshuang Zhao, Li Zhang, Shiji Song, and Gao Huang. Rt-detr: Real-time detection transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

[44] Fucheng Zheng, Duaa Zuhair Al-Hamid, Peter Han Joo Chong, Cheng Yang, and Xue Jun Li. A review of computer vision technology for football videos. *Information*, 16(5):355, 2025. doi: 10.3390/info16050355.

[45] X. Zheng and N. Liu. Color recognition of clothes based on k-means and mean shift. In *2012 IEEE International Conference on Intelligent Control, Automatic Detection and High-End Equipment (ICADE)*, pages 49–53, Beijing, China, 2012. doi: 10.1109/ICADE.2012.6330097.

[46] Yuxiao Zhou and Kecheng Yang. Exploring tensorrt to improve real-time inference for deep learning. In *2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*, pages 2011–2018, 2022. doi: 10.1109/HPCC-DSS-SmartCity-DependSys57074.2022.00299.

10 appendices